

最优化方法上机作业

叶宇涵 2100010664

2023 年 12 月 21 日

问题重述

Group Lasso 问题:

$$\min \frac{1}{2} \|Ax - b\|_F^2 + \mu \|x\|_{1,2},$$

其中

$$\|x\|_{1,2} = \sum_{i=1}^n \|x(i, 1:l)\|_2.$$

为表述方便, 令

$$g(x) = \frac{1}{2} \|Ax - b\|_F^2, \quad h(x) = \|x\|_{1,2}, \quad f(x) = g(x) + h(x).$$

我们使用多种方法求解上述问题, 比较不同方法间的性能差异。特别地, 多做了选做题 3.(b)。

完整数值结果表格

表 1: 所有结果的比较

Solver	Fval(Obj)	Sparsity	Err(Exact)	Time(s)	Iter
1 gl_cvx_mosek	5.38327E-01	0.115	4.22E-05	11.85	-
1 gl_cvx_gurobi	5.38327E-01	0.114	4.22E-05	12.22	-
2 gl_mosek	5.38315E-01	0.119	8.27E-05	19.57	-
2 gl_gurobi	5.38327E-01	0.112	4.16E-05	19.77	-
3a gl_SGD_primal	5.38327E-01	0.100	4.06E-05	7.39	8950
3b gl_GD_primal	5.38327E-01	0.100	4.06E-05	7.43	11475
3d gl_ProxGD_primal	5.38327E-01	0.100	4.06E-05	0.94	1476
3e gl_FProxGD_primal	5.38327E-01	0.100	3.99E-05	0.94	1423
3f gl_ALM_dual	5.38327E-01	0.114	4.20E-05	0.69	1207
3g gl_ADMM_dual	5.38326E-01	0.113	4.20E-05	0.19	345
3l gl_ADMM_primal	5.38348E-01	0.104	5.04E-05	0.46	2929

问题 1,2. (在 cvx 中) 使用 mosek/gurobi 直接求解

原问题可以在 cvx 中调用 mosek/gurobi 求解, 但上述格式不能直接使用 mosek/gurobi 求解, 需要改变形式:

首先把矩阵 x 按行拼接, 改写成向量形式, 即

$$x(i, j) = \tilde{x}((i-1)*l + j).$$

原问题等价于

$$\min \quad \frac{1}{2} \|\tilde{A}\tilde{x} - \tilde{b}\|_2^2 + \mu \sum_{i=1}^n \|\tilde{x}(l \times (i-1) + 1 : l \times i)\|_2.$$

引入中间变量 t, z , 改写成二次 (锥) 规划:

$$\begin{aligned} \min \quad & \frac{1}{2} \|t\|_2^2 + \mu \sum_{i=1}^n z_i \\ \text{s.t.} \quad & \tilde{A}\tilde{x} - \tilde{b} = t \\ & z_i \geq \sqrt{\sum_{k=l(i-1)+1}^{li} \tilde{x}_k^2}, \quad \forall i = 1, 2, \dots, n \end{aligned}$$

现在仍然无法用 mosek 或 gurobi 直接求解 (由于目标函数有二次项)。还需要把它转化为旋转二次锥:

$$\begin{aligned} \min \quad & \frac{1}{2} v + \mu \sum_{i=1}^n z_i, \\ \text{s.t.} \quad & \tilde{A}\tilde{x} - \tilde{b} = t, \\ & z_i \geq \sqrt{\sum_{k=l(i-1)+1}^{li} \tilde{x}_k^2}, \quad \forall i = 1, 2, \dots, n, \\ & vu \geq \|t\|_2^2, \quad u = 1, \end{aligned}$$

上述格式已经可以使用 mosek 求解。

在 gurobi 中, 二次锥约束应转化为二次约束, 并增加非负约束:

$$\begin{aligned} \min \quad & \frac{1}{2} v + \mu \sum_{i=1}^n z_i, \\ \text{s.t.} \quad & \tilde{A}\tilde{x} - \tilde{b} = t, \\ & z_i^2 \geq \sum_{k=l(i-1)+1}^{li} \tilde{x}_k^2, \quad \forall i = 1, 2, \dots, n, \\ & vu \geq \|t\|_2^2, \quad u = 1, \\ & z_i \geq 0, \quad \forall i = 1, 2, \dots, n. \end{aligned}$$

实验结果

直接调用现成函数, 结果见前文表格 1。

3(a). 次梯度法求解原问题

$g(x)$ 在任意点可微, 且

$$\begin{aligned} g(x) &= \frac{1}{2} \|Ax - b\|_F^2 \\ &= \frac{1}{2} \text{tr}((Ax - b)^T (Ax - b)) \\ &= \frac{1}{2} \text{tr}(x^T A^T Ax) - \text{tr}(b^T Ax) + \frac{1}{2} \text{tr}(b^T b). \end{aligned}$$

故 $g(x)$ 的梯度

$$\nabla g(x) = A^T Ax - A^T b.$$

$h(x)$ 在除 $\|x(i, 1:l)\|_2 = 0$ 之外的位置可微, 因此

$$\frac{\partial h(x)}{\partial x_{ij}} = \frac{x_{ij}}{\|x(i, 1:l)\|_2}.$$

当 $\|x(i, 1:l)\|_2 = 0$ 时, 函数对 x_{ij} 不可导, 且次梯度只能取 0, 因此 $h(x)$ 的次梯度 g 满足

$$g_{ij} = \begin{cases} \frac{x_{ij}}{\|x(i, 1:l)\|_2}, & \|x(i, 1:l)\|_2 \geq 0, \\ 0, & else. \end{cases}$$

由此得到, 目标函数的次梯度为 $\nabla g(x) + \mu g$ 。

算法实现细节

1. **预热策略.** 使用递减的 μ , 开始时设置 $\mu^{(0)} = 2^8 \mu$, 并进行最多 1000 步的次梯度下降。之后使得 $\mu^{(k)} = \mu^{(k-1)}/2$ 并同样进行最多 1000 步的次梯度下降。直至 $\mu = \mu^{(k)}$ 。这个思想跟**模拟退火算法的降温过程**十分类似。
2. **自适应下降步长.** 当 $\mu^{(k)} \neq \mu$ 时, 下降步长 $= 1e-3$; 当 $\mu^{(k)} = \mu$ 时, 下降步长 $= 1e-5$ 。**最初的计算下降步长较大 (保证效率), 后来的计算保证精度。**
3. **对稀疏位置的截断舍入.** 每次进行一步次梯度下降之后, 若 x_{ij} 较小, 则可以认为这一位置是 0, 我们直接将这一位置置为 0。具体实现中, 当 $\mu^{(k)} \neq \mu$ 时, 所有小于 $1e-3$ 的位置都会置为 0; 当 $\mu^{(k)} = \mu$ 时, 所有小于 $1e-5$ 的位置都会置为 0。**最初的计算允许相对较大的误差 (保证效率), 后来的计算保证精度。**

数值结果见表1。

3(b). 光滑化次梯度求解原问题

使用 Moreau envelope 方法对二范数进行光滑化。

$$\begin{aligned} M_{\|\cdot\|_2}^\alpha(x) &= \inf_z (h(z) + \frac{1}{2\alpha} \|z - x\|_2^2) \\ &= \inf_z (\|z\|_2 + \frac{1}{2\alpha} \|z - x\|_2^2) \\ &= \frac{1}{\alpha} \inf_z (\alpha \|z\|_2 + \frac{1}{2} \|z - x\|_2^2). \end{aligned}$$

z 是二范数的邻近算子:

$$z = \begin{cases} x - \frac{\alpha x}{\|x\|_2}, & \|x\|_2 \geq \alpha, \\ 0, & \|x\|_2 < \alpha. \end{cases}$$

带入上式, 得到

$$M_{\|\cdot\|_2}^\alpha(x) = \begin{cases} \|x\|_2 - \frac{1}{2}\alpha, & \|x\|_2 \geq \alpha, \\ \frac{1}{2\alpha} \|x\|_2^2, & \|x\|_2 < \alpha. \end{cases}$$

由此得到光滑化后的目标函数:

$$g(x) + \mu \sum_{i=1}^n M_{\|\cdot\|_2}^\alpha(x(i, 1:l)).$$

梯度:

$$\nabla M_{\|\cdot\|_2}^\alpha(x) = \begin{cases} \frac{1}{\|x\|_2} x, & \|x\|_2 \geq \alpha, \\ \frac{1}{\alpha} x, & \|x\|_2 < \alpha. \end{cases}$$

注意到 $x = 0$ 时上述梯度为 0, 这与 (前一问的) 次梯度法相符。

算法实现细节

1. 与前一问次梯度法使用相同技巧, 设置相同的超参数。
2. **更新光滑化参数.** 光滑化参数初始值为 $\alpha^{(0)} = 1e-2$, 每次迭代 $\mu^{(k)}$ 时, 令 $\alpha^{(k)} = \alpha^{(k-1)} - (1e-3)$ 。这样一来, 在 $\mu^{(k)} = \mu$ 时, 光滑化参数降为 0, 即不适用光滑化方法。**这相当于在最后使用次梯度法进行修正。**

实验结果分析

数值结果见表1。容易看出，此方法与次梯度法的数值精度和效率都相近（次梯度法反而稍好一些）。

3(d). 近似点梯度法求解原问题

首先考虑 $\|x\|_{1,2}$ 的近似点梯度，令 $z = \text{prox}_{t\|\cdot\|_{1,2}}$ 。根据定义，

$$z(l) = \text{prox}_{t\|\cdot\|_2}(x(l)) = \max\{0, 1 - \frac{t}{\|x(l)\|_2}\}x(l)$$

由此可以逐行定义 $\|x\|_{1,2}$ 的近似点梯度，迭代方法为

$$x^{(k+1)} = \text{prox}_{t_k \mu \|\cdot\|_{1,2}}(x^{(k)} - t_k A^T(Ax^{(k)} - b))$$

算法实现细节

与前两问类似：

1. **预热策略.** 使用递减的 μ ，开始时设置 $\mu^{(0)} = 2^{16}\mu$ ，并进行最多 100 步的次梯度下降。之后使得 $\mu^{(k)} = \mu^{(k-1)}/4$ 并同样进行最多 100 步的次梯度下降。直至 $\mu = \mu^{(k)}$ 。与之前不同的是，这里 μ 的初始值更大，下降速度更快。
2. **自适应迭代步长.** 当 $\mu^{(k)} \neq \mu$ 时，迭代步长 $\tau = 1e-3$ ；当 $\mu^{(k)} = \mu$ 时，迭代步长 $\tau = 1e-4$ 。最初的计算下降步长较大（保证效率），后来的计算保证精度。
3. **对稀疏位置的截断舍入.** 和前两问完全相同。

实验结果分析

数值结果见表1

可以看出，近似点梯度法的时间和迭代次数**远优于**之前的方法。这是因为，一方面，我们对 μ 的取值使用预热操作（热启动）；另一方面，近似点梯度法本身也更加适合原问题。

3(e).FISTA 方法求解原问题

依据上一部分的结果，我们可以推导出 FISTA 方法的迭代公式如下：

$$y = x^{(k-1)} + \frac{k-1}{k+2}(x^{(k-1)} - x^{(k-2)})$$
$$x^{(k+1)} = \text{prox}_{t_k \mu \|\cdot\|_{1,2}}(y - t_k A^T(y - b))$$

算法实现细节

除去沿原计算方向计算新点这一步之外（FISTA 算法的核心步骤），与前一问近似点梯度法使用完全相同的技巧，设置相同的超参数。

实验结果分析

数值结果见表1。FISTA 方法在实验中并没有比近似点梯度法更快，这是因为在对较大的 μ 做预处理时，已经得到了接近最优解的结果。实际上，**FISTA 算法仅在最后若干次迭代中起到了加速作用（但还能看出，总迭代次数从 1476 稍微减少至 1423）。**

3(f). 增广拉格朗日函数法求解对偶问题

接下来考虑对偶问题：

$$\begin{aligned} \min_{z \in \mathbb{R}^{m \times l}} \quad & \frac{1}{2} \cdot \|z\|_F^2 + \sum_{i,j} z_{ij} \cdot b_{ij} \\ \text{s.t.} \quad & \|A^T z\|_{\infty, 2} \leq \mu \end{aligned}$$

其中与 $\|x\|_{1,2}$ 类似， $\|x\|_{\infty, 2} = \max_i \|x(i, 1:l)\|_2$.

迭代格式

对于 ALM 子问题，我们使用**投影梯度法**迭代 100 次，格式如下：

$$v^{(k)} = \text{proj}_{\mu} \left(A^T z^{(k-1)} + \frac{\lambda}{t} \right) \quad (v \text{ 在这里是中间变量})$$

其中 $\text{proj}_{\mu}(x)$ 表示 x 在半径为 μ 的二范数球上的投影。

$$z^{(k)} = (I_m + t \cdot AA^T)^{-1} \cdot (t \cdot Av^{(k)} - A\lambda - b)$$

拉格朗日乘子的更新公式：

$$\lambda = \lambda + \sigma_k \cdot (A^T z - v)$$

σ_k 为第 k 步的罚因子。

算法实现细节

设置合适的罚因子更新系数，子问题的最大迭代次数为 100，子问题迭代步长 $t = 1e-3$ ，同样对 μ 的取值使用预热策略（与前文相同）。

实验结果分析

数值结果见表1。本方法的（外循环）迭代次数与运行时间稍优于此前的近似点梯度法与 FISTA 方法。此外，作出对偶问题与原始目标函数值与外循环迭代步数的关系图，尺度分别取 0 步后、50 步后、600 步后，得到结果如图 1,2,3。

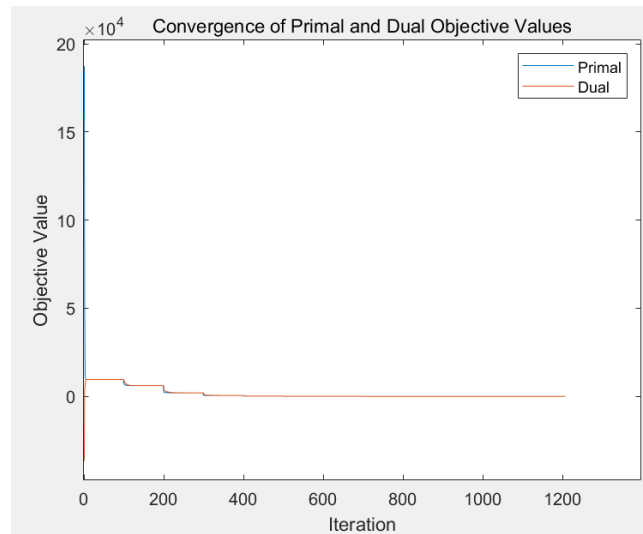


图 1: ALM_dual 迭代 0 步后

容易看出，对偶目标函数与原目标函数都呈阶梯状收敛，这主要是 μ 的下降导致的。（也可以看出，如果一开始就取问题中的 μ ，那么收敛将非常缓慢）

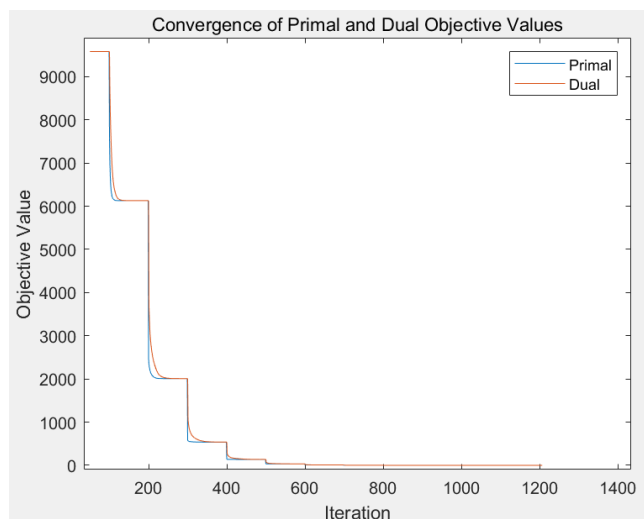


图 2: ALM_dual 迭代 50 步后

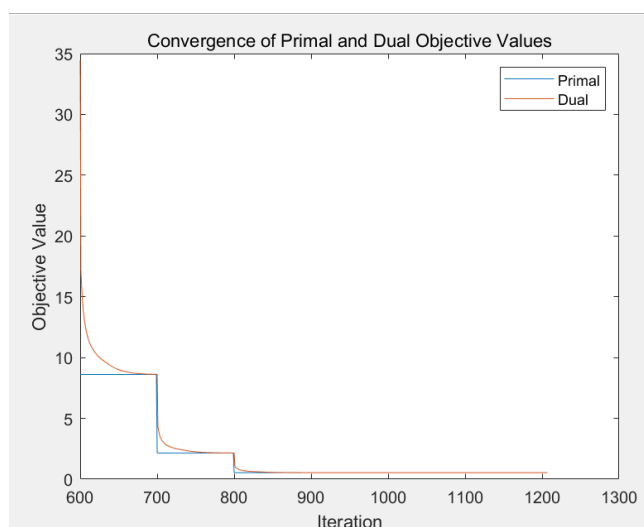


图 3: ALM_dual 迭代 600 步后

3(g). 交替方向乘子法求解对偶问题

注意到对偶问题的目标函数很容易分成两部分：

$$f_1(z) = \frac{1}{2} \cdot \|z\|_F^2$$

$$f'_2(v) = \sum_{i,j} v_{ij} \cdot b_{ij}$$

其中 $z, v \in \mathbb{R}^{m \times l}$ 。

把约束写成示性函数 $I(v) = I_{\|A^T v\|_{\infty, 2} \leq \mu}(v)$ 的形式，与 $f'_2(v)$ 相加作为 $f_2(v)$ ，再添加约束条件 $z = v$ ，即可得到适用 ADMM 求解的标准形式。

迭代格式

对偶问题的 ADMM 迭代格式如下：

1. 第一部分变量 z 的更新：

$$z = (I_m + t \cdot AA^T)^{-1} \cdot (t \cdot Av - A\lambda - b)$$

2. 第二部分变量 v 的更新:

$$v = \text{proj}_\mu \left(A^T z + \frac{\lambda}{t} \right)$$

其中 $\text{proj}_\mu(x)$ 表示 x 在半径为 μ 的二范数球上的投影。

3. 拉格朗日乘子 λ 的更新:

$$\lambda = \lambda + t \cdot (A^T z - v)$$

算法实现细节

取迭代步长 $t = 0.1$ (步长 t 的选择对实验结果的影响很大), 使用 μ 的热启动策略 (效果不明显, 故只在前 20 次迭代步使用), 取停机条件为 $1e-3$ (对偶函数收敛度阈值)。

实验结果分析

数值结果见表1。可以看出, **ADMM 的收敛速度快、迭代次数少, 与之前的方法相比有明显优越性。**

作出对偶问题与原始目标函数值与外循环迭代步数的关系图, 尺度分别取 0 步后、50 步后、120 步后, 得到结果如图 4,5,6。

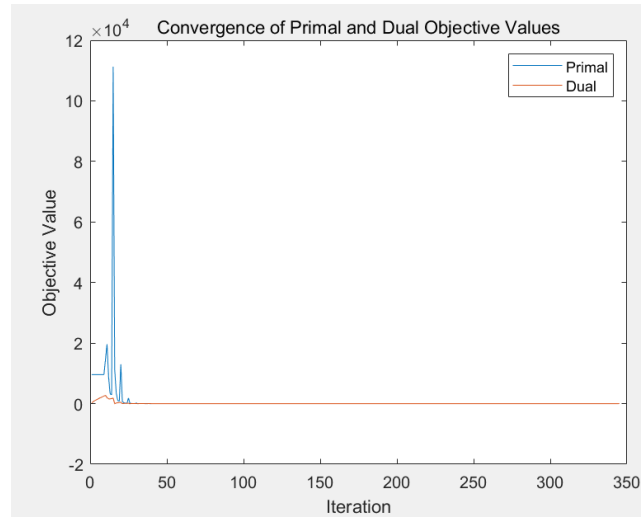


图 4: ADMM_dual 迭代 0 步后

容易看出, 原始目标函数值在迭代开始时剧烈震荡, 随后迅速收敛 (图 4); 对偶目标函数值在收敛前呈周期性震荡, 收敛速度较均匀 (图 5,6)。

3(h). 线性化交替方向乘子法求解原问题

对原问题做拆分:

$$\begin{aligned} & \underset{z, y}{\text{minimize}} && Az - \mu \|y\|_{1,2} \\ & \text{s.t.} && x - y = 0 \end{aligned} \tag{1}$$

写出增广拉格朗日函数:

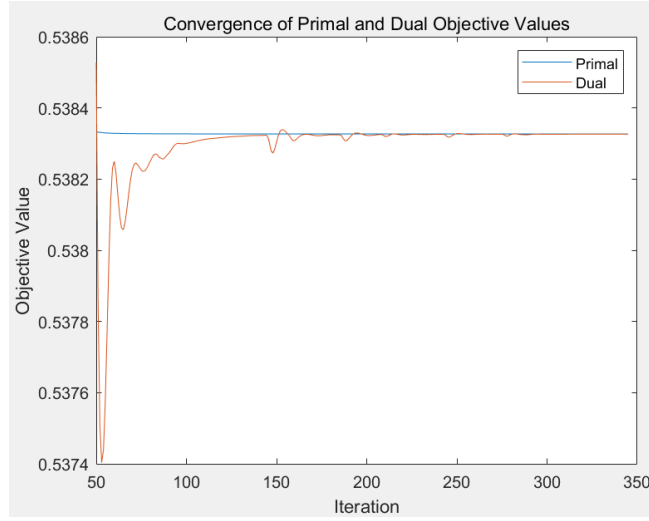


图 5: ADMM_dual 迭代 50 步后

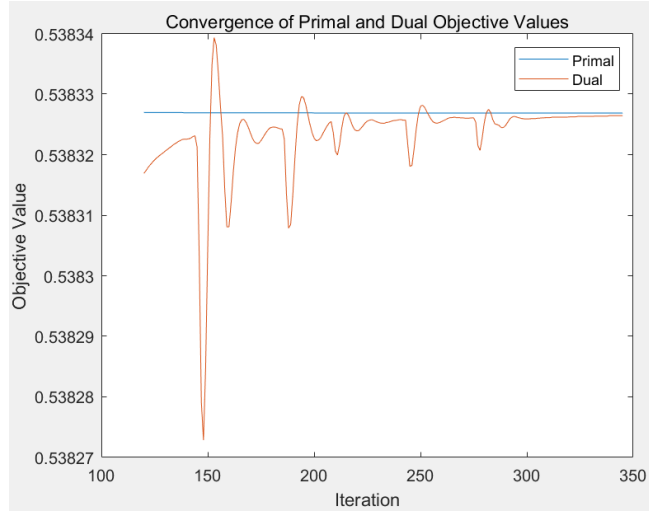


图 6: ADMM_dual 迭代 120 步后

$$L(z, y, \lambda) = Az - \mu \|y\|_{1,2} + \lambda^T (x - y) \quad (2)$$

对 x 的子问题做线性化:

$$x_{k+1} = \arg \min_x \left\{ A^T (Az_k - b) + (x - x_k) + \frac{\rho}{2} \|x - x_k\|_2^2 \right\} \quad (3)$$

这等价于对 x 做一步步长为 ρ 的梯度下降:

$$z_{k+1} = z_k - \eta (A^T (Ax_k - b) + \rho(x_k - y_k) + z_k) \quad (4)$$

对 y 的子问题做线性化 (仅对可微部分):

$$y_{k+1} = \arg \min_y \left\{ \mu \|y\|_{1,2} + \frac{\rho}{2} \|y - y_k + x_{k+1} - z_{k+1}\|_2^2 \right\} \quad (5)$$

这等价于对 y 做一步步长为 ρ 的近似点梯度下降。

算法实现细节

取迭代步长 $t = 0.1$, 固定 μ 的取值, 取停机条件为 $1e-4$ (原始函数收敛度阈值)。

实验结果分析

数值结果见表1。结合目标函数的收敛图像，可以看出收敛性状具有一定的阶梯性，这是 ADMM 本身的性质导致的（这一问不涉及 μ 的预热操作）。线性化过程使用近似点项对子问题进行二阶近似，虽然降低了计算矩阵逆的开销，但也降低了每一步迭代的精确性，因此总迭代次数有所增加，运行时间比前一问有所增长。